ReACKed QUICer: Measuring the Performance of Instant Acknowledgments in QUIC Handshakes

Jonas Mücke
jonas.muecke@tu-dresden.de
TU Dresden
Dresden, Germany

Marcin Nawrocki
marcin.nawrocki@netscout.com
NETSCOUT
Westford, MA, USA

Raphael Hiesgen
raphael.hiesgen@haw-hamburg.de
HAW Hamburg
Hamburg, Germany

Thomas C. Schmidt
t.schmidt@haw-hamburg.de
HAW Hamburg
Hamburg, Germany

Matthias Wählisch
m.waehlisch@tu-dresden.de
TU Dresden
Dresden, Germany

Abstract

In this paper, we present a detailed performance analysis of QUIC instant ACK, a standard-compliant approach to reduce waiting times during the QUIC connection setup in common CDN deployments. To understand the root causes of the performance properties, we combine numerical analysis and the emulation of eight QUIC implementations using the QUIC Interop Runner. Our experiments comprehensively cover packet loss and non-loss scenarios, different round trip times, and TLS certificate sizes. To clarify instant ACK deployments in the wild, we conduct active measurements of 1M popular domain names. For almost all domain names under control of Cloudflare, Cloudflare uses instant ACK, which in fact improves performance. We also find, however, that instant ACK may lead to unnecessary retransmissions or longer waiting times under some network conditions, raising awareness of drawbacks of instant ACK in the future.

CCS Concepts

• **Networks** → **Transport protocols**; **Network protocol design**; *Network performance evaluation.*

Keywords

QUIC; spurious retransmissions; HTTP/3; CDN

ACM Reference Format:
Jonas Mücke, Marcin Nawrocki, Raphael Hiesgen, Thomas C. Schmidt, and Matthias Wählisch. 2024. ReACKed QUICer: Measuring the Performance of Instant Acknowledgments in QUIC Handshakes. In *Proceedings of the 2024 ACM Internet Measurement Conference (IMC '24), November 4–6, 2024, Madrid, Spain.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3646547.3689022

1 Introduction

Low latencies are key for Content Delivery Networks (CDNs), even if savings are in the range of few milliseconds. With handshakes often spanning multiple round trip times (RTTs) between client and CDN server, a reduction of this latency is beneficial. QUIC [7] promises such a reduction by combining the transport and encryption (TLS) handshakes to establish a connection in a single round trip time (1-RTT). This makes QUIC popular in large-scale content provider setups [19, 25, 30] and recent protocols, *e.g.,* HTTP/3 [1].

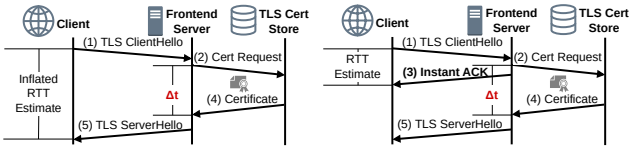
Common CDN deployments challenge a fast connection setup in QUIC. During the connection setup in QUIC, the server requires a TLS certificate to answer the client connection request. In many CDN setups, the server needs to fetch this certificate from a backend server. Waiting until the certificate is available inflates the RTT. Alternatively, the server could acknowledge the client request instantly and continue the handshake after the certificate is received. We dub this behavior *instant ACK*. An instant ACK promises a more realistic RTT estimation between server and client, and therefore leads to more realistic timeouts in case of packet losses. Cloudflare, a major CDN, implements this behavior [18]. Whether an instant ACK is an advantageous option during the QUIC connection setup is an open topic, though.

In this paper, we empirically analyze the performance of instant ACK in multiple scenarios and compare with deployments in which a server waits until the certificate is available. Our analysis allows us to conclude when an instant ACK is advantageous, how much improvement can be achieved in real deployments, and whether further adoption of instant ACK would be beneficial to reduce connection setup latencies. Furthermore, we study the adoption of instant ACK in CDNs and measure improvements within one major CDN. In detail, our contributions are as follows:

(1) We analyze the effects of instant ACK, finding improvements and deterioration. Probe Timeouts (PTOs) are improved by 3× the delay between frontend server and certificate store. On packet loss, recovery is faster – in the order of 10 ms in the wild. When QUIC servers are blocked due to the anti-amplification limit, instant ACK may improve handshake completion by up to 1 RTT.
(2) We find that the instant ACK induces spurious retransmissions in scenarios of high latency between frontend server and certificate store.
(3) We measure the global deployment of instant ACK. Up to 99.9 % of Cloudflare, 41 % of Amazon, 32.2 % of Akamai, and 11.5 % of Google domains on the Tranco Top 1M use instant ACK.

This work is licensed under a Creative Commons Attribution International 4.0 License.

IMC '24, November 4–6, 2024, Madrid, Spain
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0592-2/24/11
https://doi.org/10.1145/3646547.3689022



(a) Wait for certificate (WFC) (b) Instant ACK (IACK)

Figure 1: QUIC server behaviors in common CDN deployments. The delay Δt between frontend server and certificate store inflates the RTT estimate for subsequent data from the frontend server, if the frontend server does not use IACK.

(4) We analyze in which scenarios QUIC connections benefit most from the instant ACK optimization and show that the Cloudflare deployment lies within that range. We identify tuning parameters for client and server configurations.
(5) We conclude from comparing different implementations and from numerical analysis that improvements are not implementation dependent but on the protocol level. We also find inconsistent client behavior and work with the maintainers on resolving this.

2 Background and Related Work

Figure 1 illustrates a simplified QUIC handshake in common CDN deployments. In response to the TLS ClientHello, the server needs encryption keys to compile the TLS ServerHello, certificate and signatures, and returns them to the client. Large CDNs, such as Akamai and Cloudflare, separate TLS endpoints and certificate stores [2, 25] to liberate TLS endpoints from storing all customer certificates. Certificates are provisioned on the frontend server when needed, often after a connection request. The frontend server then has two options. (*i*) The server waits until the certificate is retrieved from the certificate store, we dub this *wait for certificate (WFC)*, or (*ii*) replies immediately with an ACK to the client and delivers the ServerHello as soon as the certificate is available, we dub this *instant ACK (IACK)*.

Calculation of the Probe Timeout. The RTT estimation determines the Probe Timeout (PTO). The PTO is initialized with 3× of the first RTT sample. Although ACKs include an acknowledgment delay to correct RTT estimation of intentionally delayed ACKs, the PTO initialization disregards this delay. Therefore, the only option to provide the client with an accurate PTO is via the instant ACK. Subsequently, the PTO is based on the exponentially weighted moving averages of the smoothed RTT plus RTT variation, and the maximum acknowledgment delay provided by each endpoint. A PTO timer is set and reset when a packet eliciting an ACK is sent or acknowledged and when Initial or Handshake keys are discarded. Whenever a PTO expires the sender must send 1-2 datagram probes, which elicit an ACK and exponentially backoff the PTO. Retransmitting tail bytes in-flight is recommended; they can be bundled with outstanding data if available. If neither are available, senders will fall back to sending PING frames, which do not carry further information. Sending acknowledgments, *e.g.,* in response to PING frames, does not cause a PTO reset.

Potential disadvantages of wait for certificate. When a server waits for the certificate before responding to a client, the handshake is delayed, see Δt in Figure 1(a). This delay impacts the Round Trip Time (RTT) estimation by the client. The client then assumes an RTT that is inflated compared to the RTT between the client and frontend server, which delivers the web content.

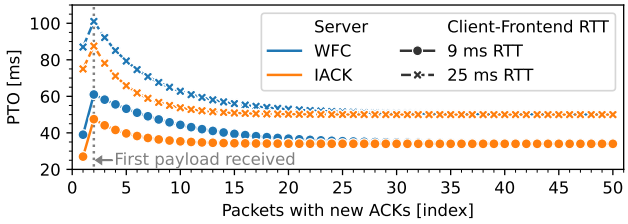


Figure 2: Calculated evolution of the Probe Timeout (PTO) assuming that all subsequent packets arrive exactly after one RTT and the instant ACK is delivered 4 ms earlier. The instant ACK leads to a PTO improvement of $3 \times \Delta t$.

Potential advantages of instant ACK. The instant ACK reduces the PTO and consequently the time until retransmissions occur. This is important (*i*) under packet loss and (*ii*) during the handshake, when the server is limited by the QUIC anti-amplification limit. We illustrate the evolution of the PTO in a static setting in Figure 2. An inflated PTO slowly approaches the actual PTO between client and frontend server, resulting in negative effects for content delivery in case of packet loss. However, when the delay between the frontend server and certificate store is larger than the first instant ACK PTO, this leads to spurious retransmits.

To avoid amplification attacks, the server is limited to send 3× the data received from the client until the client address is verified [7]. If the handshake exceeds this limit, the server needs to wait for additional client data to increase its amplification budget. In this situation, earlier PTO expiration leads to earlier probe packets, which accelerate the QUIC handshake even without packet loss. Earlier work showed that Cloudflare implements instant ACKs [18], but we also find that other CDNs experiment with instant ACKs.

Related work. Analyzing QUIC performance is an ongoing research field. Flow control, multiplexing strategies [12, 29], robustness [17], resource prioritization [21], CPU [8] or NIC [28] architectures, and the congestion control (CC) algorithm [4, 15, 16, 27] impact performance. Short-lived connections benefit most from QUIC [24, 29], and QUIC shows better performance [20, 23, 27, 29] in the presence of lossy links. Compared to TCP, QUIC improves RTT estimation [24, 29] due to simplified packet numbering. Default host buffer sizes, however, are too small by an order of magnitude [8] to achieve high goodput, and current TLS certificate configurations prevent 1-RTT handshakes [18]. Prior work relies on random packet drop rates or selected deployments, including IoT scenarios [3], satellite communication [5, 9], and cellular networks [4, 16], as well as interoperability testing [22]. The combination of client and server implementations lead to different performance expectations [8].

We extend the field by analyzing the performance impact of instant ACK in a well-known testbed and a common CDN setup. Our emulation comprises multiple QUIC implementations, RTTs ranging from local to long distance, and distinct packet drops instead of relying on stochastic or exemplary deployments. This allows pinpointing the effects of instant ACK.

3 Measurement Method and Setup

To compare the impact of instant ACK on QUIC handshakes under conditions of packet loss, detailed information on the exchanged packets, their timing, and the experienced packet loss are required. We rely on a controlled testbed to obtain this information. We measure instant ACK deployments by performing QUIC handshakes with several servers in the wild and analyzing response traffic. Based on the collected data we calculate PTOs and compare behaviors.

Microscopic view—Testbed. We emulate network conditions using the QUIC Interop Runner (QIR), a container-based framework for interoperability testing of QUIC implementations [22]. QIR captures packets and collects Qlog information from clients and servers. Qlog [14], a structured logging format for QUIC, contains data about sent packets, received packets, and *recovery:metrics*, including the smoothed RTT and RTT variation calculated by the implementation. Nonetheless, implementations differ in how often and how exhaustive *recovery:metrics* are exposed (see Appendix E). Writing Qlogs might slow down implementations [8]. The exposed information, however, is the best approximation of a systems view on connections since delays cover the networking stack, scheduling, and abstraction layers—similar to real-world deployments and more detailed than packet captures. Qlog is commonly used for such analyses [12, 13, 29]. Multiple repetitions help us to identify patterns. To ensure consistency, we calculate PTOs based on sent and received packets according to the standard [6].

Our evaluations consist of an HTTP `GET` request and response. We test the transfer of 10 KB and 10 MB randomly generated files, as representatives for an embedded library and larger websites.

The server is armed with two certificates: one allows a 1-RTT handshake (1,212 B), the other (5,113 B) exceeds the anti-amplification limit. Backend–frontend delays are emulated by a configurable sleep period in the server code. All simulations use a bandwidth of 10 Mbit/s. RTTs are composed of symmetric one way delays between 0.5 ms and 150 ms, based on typical RTTs measured in different regions of the world [11]. Every test is repeated 100 times.

Packet loss is often modeled as a random share of lost packets or observed on real world links. Our emulation instead simulates particular datagram losses to better understand root causes of specific performance properties. The QUIC protocol allows coalescence of multiple QUIC packets into a single UDP datagram, *i.e.,* an entire flight can be transmitted in one datagram (see Figure 3). Implementations use this feature to different extents. Consequently, the impact of a single packet loss differs between implementations. Unless stated otherwise, we match lost datagrams to their QUIC content and compare equal information loss. This ensures that different packet coalescence does not influence protocol level conclusions.

We perform QUIC connections from eight client implementations (*aioquic, go-x-net, mvfst, neqo, ngtcp2, picoquic, quic-go, quiche*) to a *quic-go* server modified to support IACK. Clients request data via HTTP/1.1 and HTTP/3 (except *go-x-net*, which does not implement HTTP/3). Although Cloudflare is reported to implement instant ACK, the public source code for *quiche* does not contain IACK. The simulations are distributed over 18 test runners (virtual machines with 4 AMD EPYC 7513 Cores, 4 GB RAM, Fedora 39). Virtualization and containerization are a common practice in industry. To ensure that these abstractions do not bias our results, we verify them by thoroughly inspecting packet captures and tracing differences back to the protocol behavior.

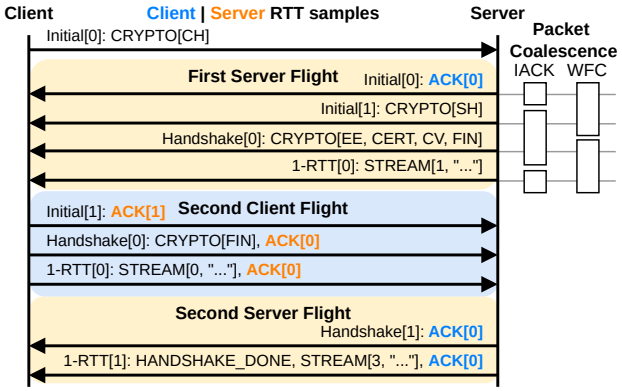


Figure 3: QUIC 1-RTT connection setup [7] and coalescence differences between IACK and WFC. Blue and orange ACKs indicate client and server RTT samples.

Macroscopic view—Instant ACK deployments. We perform QUIC handshakes and HTTP/3 `HEAD` requests using QScanner [30]. We target 1M domain names in the Tranco toplist [10] from August 06, 2024 and repeat our measurements on three subsequent days. We then map the contacted IP addresses to ASes and on-net CDN deployments (see Appendix G).

To consider side effects introduced by third parties (*e.g.,* caching of certificates on frontend servers), we add twelve of our own otherwise unused domains to the Cloudflare Free Tier CDN with default settings. The TLS certificates of those domains are managed by Cloudflare and signed by Google Trust Services. For comparison, we select six domains from the Top 1000 Tranco domains that are served by Cloudflare. We then schedule one connection per minute to six of our domains and the six domains selected from Tranco. To our remaining six domains, we schedule 60 connections per minute. We run these measurements for one week, collect all response traffic, and analyze the content using a packet dissector. The location of the replying server is derived from the IATA location identifier included in the HTTP header `Cf-Ray`, which is provided by Cloudflare servers. We only consider responses from servers located in the same city as our measurement probe to eliminate larger network effects and ensure that we reach the same cluster. Additionally, we account for packet loss by including only responses that contained the first ACK of the connection. We compare the request and arrival times of the received ACK, ServerHello, and coalesced ACK–ServerHello.

To cross-verify our results, we perform all measurements from a European university network (Hamburg, DE) and Google Cloud VMs in North America (Los Angeles, US), South America (Sao Paulo, BR), and Asia (Hong Kong, HK) in August 2024.

4 Results

In this section, we (*i*) assess when an instant ACK yields most benefits, (*ii*) measure the performance impact of instant ACK in QUIC implementations with and without packet loss, and (*iii*) analyze instant ACK in global CDN deployments.

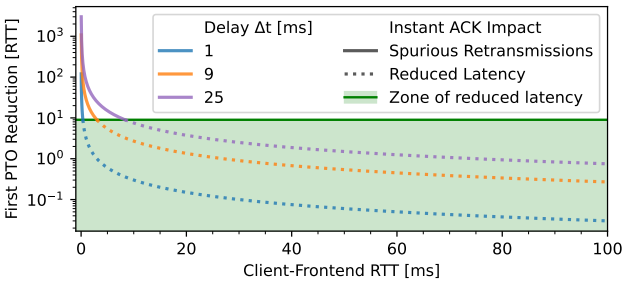
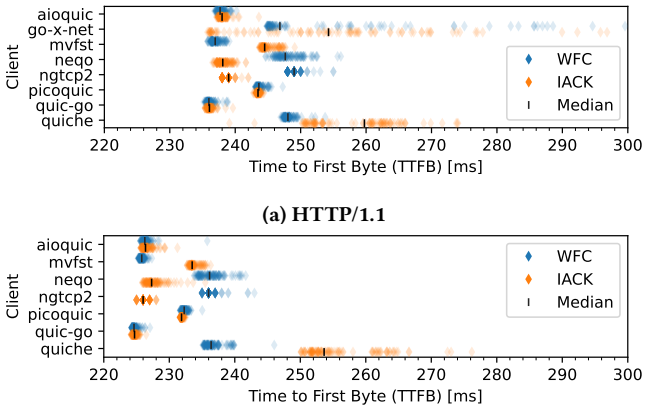


Figure 4: First PTO improvement according to RFC9002 [6]. Spurious retransmits happen if the delay between Frontend Server and Cert Store (Δt) is larger than the PTO set by the client. Relative to the RTT, lower latency connections profit more from PTO improvement with IACK.



(a) HTTP/1.1

(b) HTTP/3

Figure 5: Time to First Byte (TTFB) of 10 KB file transfer at 9 ms RTT with large certificate, $\Delta t = 200$ *ms*, and without packet loss. Each diamond symbol represents the TTFB for an individual measurement. Intense colors indicate a concentration of values. HTTP/3 generally has a lower TTFB because the first STREAM frame received from the server is the Control Stream with the SETTINGS frame, which is send by the server immediately after the handshake completes. Compared to HTTP/1.1, this is one RTT faster.

4.1 Baseline Performance

Numerical sweet spot analysis. Before we present results based on our testbed, we use a numerical analysis to highlight the principle impact of instant ACK. Figure 4 shows the theoretical PTO reduction relative to the RTT and the area in which no spurious retransmissions occur for different Δt and varying RTTs.

Instant ACK improves performance in two scenarios. First, in case of client packet loss, the shorter PTO is beneficial because the client repeats its requests earlier. The maximum PTO improvement is achieved when Δt approaches the client PTO. Second, when Δt exceeds the client PTO and the server is blocked by the anti-amplification limit, the client will send probe packets earlier, increasing the amplification budget leading to a faster termination of the handshake. If the server is not blocked during the handshake instant ACK might harm because the client probing due to expired PTO causes additional load on the server without any benefit.

QUIC stack delays. The Δt does not only consist of the network delay between the frontend server and certificate store but also of the local processing time. In our testbed, we measured that the delay difference between IACK and WFC reported by the different QUIC clients is on median 2.9 ms (*neqo*) to 7.8 ms (*mvfst*), except for *go-x-net*. *go-x-net* introduces high variations in individual measurements (median 0.1 ms to 12.7 ms) and partly reports erroneous values.

At the server side, to send an instant ACK, the QUIC stack needs to calculate the Initial keys based on information from the client packet. In case of a coalesced message, additionally, handshake keys need to be calculated and TLS ServerHello, certificate, and signatures compiled. Profiling 100,000 handshakes of our server implementation *quic-go* reveals that signature calculation is the single most CPU consuming function.

Unless specified otherwise, we do not add delays in our emulations.

Instant ACK improves the first PTO. Our measurements confirm that instant IACK improvements are independent of the RTT for all client implementations. Only if the default PTO is lower than the RTT, side effects can occur (see Appendix E). In a network with lower latencies, the same improvement would be relatively more beneficial (see Figure 4). This aligns with the theoretically possible improvement, although accuracy in practice is limited by the information exposed in Qlog (see Appendix E). Accounting for the influence of the default PTO and erroneous PTO calculation, the median PTO improvement between IACK and WFC for 100 measurements per RTT (1 ms …300 ms) ranges between 7 ms and 24.7 ms. Given that RTTs between end-users and CDN servers are typically in the same range [26] such an improvement is significant.

Time to First Byte (TTFB) is reduced if the server is blocked by the anti-amplification limit. When the TLS certificate size exceeds the anti-amplification limit the server must wait for additional packets from the client (*e.g.*, because the PTO expires). The client PTO includes the client-frontend RTT. In a lossless scenario, the next packet will then be delivered later than $3 \times RTT$. We emulate this case using a certificate (size 5113 B) that exceeds the anti-amplification limit and a 200 ms delay before sending the ServerHello, *i.e.*, the first server packet in WFC and the second one in IACK. Figure 5 shows a decreased median TTFB with most significant improvements for *neqo* (9.6 ms) and *ngtcp2* (10 ms). The server logs confirm that WFC is more likely to cause the server to block due to the anti-amplification limit, which happens significantly less when instant ACK is enabled. In *go-x-net*, the client partially initializes the smoothed RTT and RTT variation incorrectly, which leads to worse performance (*e.g.*, reported RTT 33 ms, but smoothed RTT is initialized at 90 ms). However, if the RTT estimates are initialized correctly, IACK improves the TTFB. In *mvfst and picoquic*, receiving an instant ACK does not cause the client to send probe packets. For picoquic, the default server PTO is reached first, leading to equal performance of IACK and WFC. In *aioquic*, *mvfst*, and *quic-go*, the default client PTO expires before reception of the coalesced ACK–ServerHello and increases the amplification budget in both WFC and IACK before the server sends the ServerHello. Using *quiche*, we observe negative effects when IACK is enabled because the *quiche*

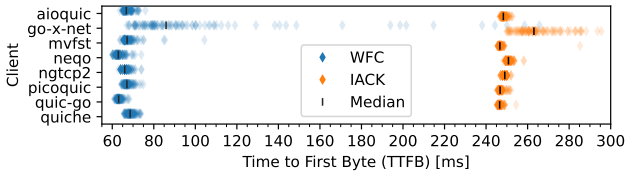


Figure 6: Time to First Byte of 10 KB file transfer at 9ms RTT under loss of packets 2 and 3 (IACK) and packet 2 (WFC) sent by the server. IACK prolongs the TTFB.

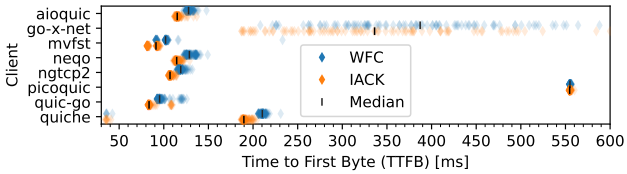


Figure 7: Time to First Byte of 10 KB file transfer at 9 ms RTT under loss of the entire second client flight (see Appendix E). Instant ACK improves the TTFB.

client drops replies to PING frames as invalid together with coalesced packets. This then requires retransmission of the dropped information.

4.2 Packet Loss Scenarios

Time to First Byte is increased when the remaining first server flight is lost. We now simulate loss of the second and third UDP datagram (IACK) and loss of the second UDP datagram (WFC) in the first server flight (*cf.,* Figure 3). In this scenario, WFC outperforms IACK (see Figure 6). IACK requires between ≈177 ms (*go-x-net*) and 188 ms (*neqo*) more time until the first payload data is received. This might appear surprising but relates to the QUIC protocol mechanics. Not every ACK leads to a new RTT sample at the receiving side—only ACKs to ACK-eliciting packets do. The IACK sent by the server is not ACK-eliciting. In consequence, when the server receives the probe packet from the client, the server will not update its RTT estimate but relies on the default PTO. In WFC, the server is able to update an RTT estimate, because the first server packet combines ACK–ServerHello and a ServerHello must be acknowledged by the client. Since the PTO based on the RTT estimate is smaller than the default PTO of the server, the retransmit and thus finishing the handshake is faster in WFC than in IACK.

Our results show that *quiche* does not respond to an IACK. This is due to an uncommon behavior of *quiche*: it drops connections when the same connection ID is retired multiple times. In our HTTP/3 measurements, we do not encounter this case. Here, *quiche* behaves like all other implementations (see Appendix F).

Time to First Byte is reduced when the second client flight is lost. Figure 7 compares TTFB between WFC and IACK when the second client flight is lost (*cf.,* Figure 3). The second client flight includes the first payload data a client can send in a 1-RTT handshake. This payload includes the HTTP request. If this packet is lost, a smaller PTO due to instant ACK will allow the client to resend the lost information sooner, leading to an improved TTFB. Independent of the simulated RTT in lossy scenarios, the first byte is received significantly sooner with on median 10 ms (*mvfst*), 11 ms (*aioquic, quic-go*), 12 ms (*neqo, ngtcp2*), 23 ms (*quiche*), and 28 ms (*go-x-net*). *picoquic* does not benefit from an earlier retransmission because it ignores the lower RTT induced by IACK.

This confirms that the improvement of the instant ACK is constant across RTTs, but the relative impact is larger for short RTTs, *e.g., quiche* improved by 2.3 RTTs under 9 ms network latency.

Table 1: Domains from the Tranco Top 1M hosted by CDNs, share of instant ACK deployment, and maximum difference between measurements. Deployment share and maximum variation are aggregated across vantage points and repetitions. Group "Others" represents hosting services.

		IACK deployment enabled (max.)	
CDN	Domains [#]	Domains [%]	Variation [%]
Akamai	533	32.2	12.9
Amazon	4338	41.0	18.0
Cloudflare	247407	99.9	0.1
Fastly	3960	0.0	0.0
Google	6062	11.5	11.5
Meta	112	0.0	0.0
Microsoft	34	0.0	0.0
Others	26404	21.5	2.3

4.3 CDN Deployment

We attempt to establish QUIC connections to all domains in the Tranco Top 1M list. If successful, we check for instant ACK behavior, *i.e.,* whether the ClientHello is followed by a separate (server) ACK preceding the TLS ServerHello.

Several CDNs support IACK. Fastly, Meta, and Microsoft do not deploy instant ACK but the majority of domains (99.9%) operated by Cloudflare do (see Table 1). Akamai, Amazon, and Google also deploy IACK but at relatively smaller scale (32.2%, 41.0%, and 11.5% of the domains). We observe the highest variations in IACK deployment from Amazon, with 8.3% difference across measurements at the same location and 18.0% across all measurements and vantage points. Figure 8 shows the delay between ACK and ServerHello. On median, the IACKs arrive 3.2 ms (Cloudflare), 6.4 ms (Amazon), 30.3 ms (Google), and 20.9 ms (Akamai) earlier than the ServerHellos across all vantage points.

The instant ACK deployment at Cloudflare. We now focus on Cloudflare since they have the largest deployment of instant ACK enabled services. To understand the impact of potential certificate caching on frontend servers, we compare handshake behavior of domains all served by Cloudflare but with different popularity (setup see § 3). Among the selected Tranco domains, we observe coalesced ACKs and ServerHellos from *discord.com (91.9% of the responses), cloudflare.com (50.5%), tinyurl.com (17.7%)*, and *docker.com (0.7%).* From *udemy.com* and *kickstarter.com*, we receive instant ACKs but no SHs follow. From our own domains targeted at the same rate

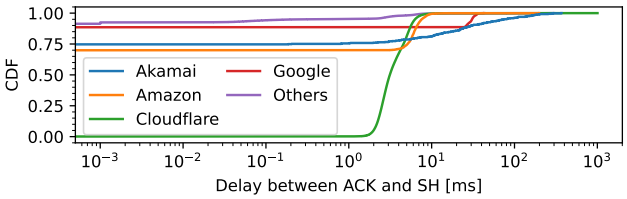


Figure 8: Delay between reception of the first ACK and subsequent ServerHello (SH) from our vantage point in Sao Paulo. Coalesced ACK–SH is shown as 0 delay. Akamai is significantly slower than other CDNs to deliver the ServerHello.

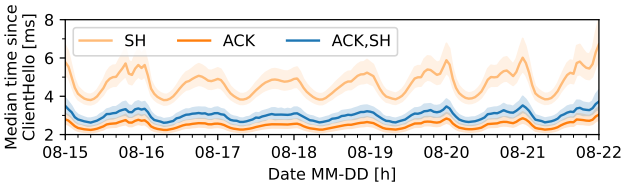


Figure 9: Reception latency and 50 % percentile interval of ACK and SH, either separately in sequential packets or coalesced ACK–SH from Cloudflare in Sao Paulo, BR. SH in coalesced messages arrive faster than separate SH.

as the Tranco domains, we almost always (99.9%) receive an instant ACK. When contacting six of our domains faster, we receive coalesced ACKs and ServerHellos more likely (7.5%).

Figure 9 illustrates the behavior for Sao Paulo. Coalesced ACKs and ServerHellos are received with similar delay compared to instant ACK, a strong indicator for caching. On median the instant ACK arrives after 2.1 ms (Sao Paulo, Hamburg), 2.4 ms (Los Angeles), and 2.6 ms before the SH (Hong Kong). All vantage points show larger delays between IACK and ServerHello during local day time compared to the night (see Appendix G). The observed delays are even smaller than the delays when connecting to all QUIC domains from the Tranco List (see Figure 8). If the server had waited for the certificate from the certificate store instead, the PTO would have been inflated by 6.3 to 7.2 ms (up to 79% of the median RTT). With this improvement and the measured RTTs, Cloudflare does not experience spurious retransmits and benefits from IACK.

5 Discussion

Is instant ACK deployment only beneficial to CDNs with separate certificate store? No. There are multiple reasons for delays in server responses. Traversing a storage hierarchy, slow database lookups for configuration, high CPU utilization, and busy cryptography units are some examples that can cause belated responses from servers inflating the RTT estimate. In our setup, we found that responses with a (simple) certificate store operated on the frontend server may take longer than an IACK, mainly caused by the TLS signing function.

When does an instant ACK harm performance? If the RTT between frontend and backend server is larger than ≈3× RTT between client and frontend server, an instant ACK will lead to spurious retransmits. Our analysis showed that these spurious retransmits may help when the server reaches the QUIC amplification limit, since the client packet (as a reply to the server instant ACK) provides the server additional sending budget. Using a padded instant ACK to probe the path MTU, as Cloudflare implements, needs careful consideration, though, since this consumes additional amplification budget, which can lead to an overall longer time until the handshake completes. If a server is not stalled by the anti-amplification limit, the probe packets add futile load to the network and server.

Can server operators rely on client implementations to correctly handle instant ACKs? No. We found that multiple implementations processed instant ACKs incorrectly, which added delays. We work with the maintainers on a resolution of the issues.

How to improve instant ACK? Server and client behavior can be tuned to achieve better performance using instant ACK. When an instant ACK was received successfully but the ServerHello and additional packets of the handshake are lost, the server has to wait until its default PTO expires. Lowering this value is a trade-off between faster recovery from packet loss and inducing spurious retransmissions. Alternatively, clients can cause the server to recover from this packet loss faster by retransmitting the ClientHello (when the PTO expires) instead of sending PING frames [6].

Generalization to 0-RTT and Retry handshakes. An instant ACK can also be used in case of 0-RTT handshakes to prevent PTO inflation. Independently of the type of handshake, the client sends a ClientHello, which may trigger an instant ACK. The reception of a Retry packet causes the client to resend the ClientHello together with a token that proves its reception. Retry is implemented to counter resource exhaustion attacks. When a client receives the Retry packet it may use this packet as the first RTT estimate. A subsequent instant ACK is still beneficial as it reduces RTT variation.

Do alternatives to instant ACK exist? Since the PTO initialization disregards the acknowledgment delay, IACK is the only mean to provide exact RTT estimates to the client at the beginning of a connection. A client could respect the acknowledgment delay at PTO initialization or reinitialize the PTO with the value of a subsequent, potentially undelayed RTT sample but the benefits are limited to subsequent exchanges or depend on correct server information (see Appendix D).

6 Conclusions

In this paper, we discussed and empirically analyzed instant ACK, a standard compliant approach to improve the QUIC connection setup in common CDN deployments. Instant ACK aims to better estimate the RTT between client and frontend server to adjust expiration timers more precisely—a fundamental challenge to reduce delays and prevent spurious retransmits seen in many transport protocols. Our measurements of popular domains revealed that Cloudflare, Akamai, Amazon, and Google deploy instant ACK in a significant share of domains. Depending on the scenario, instant ACK proved beneficial or harmful. At least one client implementation was negatively affected because it could not handle instant ACKs. We conclude that the protocol mechanics of QUIC challenge consistent performance improvement and provide guidelines under which conditions instant ACK should be enabled in Appendix C.

References

[1] M. Bishop. 2022. *HTTP/3.* RFC 9114. IETF. https://doi.org/10.17487/RFC9114

[2] Laurent Chuat, AbdelRahman Abdou, Ralf Sasse, Christoph Sprenger, David Basin, and Adrian Perrig. 2020. SoK: Delegation and Revocation, the Missing Links in the Web's Chain of Trust. In *Proc. of EuroS&P*. IEEE, New York, NY, USA, 624–638. https://doi.org/10.1109/EuroSP48549.2020.00046

[3] Jasenka Dizdarević and Admela Jukan. 2021. Experimental Benchmarking of HTTP/QUIC Protocol in IoT Cloud/Edge Continuum. In *Proc. of the IEEE International Conference on Communications*. IEEE, New York, NY, USA, 1–6. https://doi.org/10.1109/ICC42927.2021.9500675

[4] Habtegebreil Haile, Karl-Johan Grinnemo, Simone Ferlin, Per Hurtig, and Anna Brunstrom. 2022. Performance of QUIC Congestion Control Algorithms in 5G Networks. In *Proc. of ACM SIGCOMM 5G-MeMU*. ACM, New York, NY, USA, 15–21. https://doi.org/10.1145/3538394.3546040

[5] Cristina Hervella, Luis Diez, Fátima Fernández, Néstor J. Hernández Marcano, Rune Hylsberg Jacobsen, and Ramón Agüero. 2022. Realistic Assessment of Transport Protocols Performance over LEO-Based Communications. In *Proc. of the ACM PE-WASUN* (Montreal, Quebec, Canada) *(PE-WASUN '22)*. ACM, New York, NY, USA, 91–98. https://doi.org/10.1145/3551663.3558680

[6] J. Iyengar and I. Swett. 2021. *QUIC Loss Detection and Congestion Control.* RFC 9002. IETF. https://doi.org/10.17487/RFC9002

[7] J. Iyengar and M. Thomson. 2021. *QUIC: A UDP-Based Multiplexed and Secure Transport.* RFC 9000. IETF. https://doi.org/10.17487/RFC9000

[8] Benedikt Jaeger, Johannes Zirngibl, Marcel Kempf, Kevin Ploch, and Georg Carle. 2023. QUIC on the Highway: Evaluating Performance on High-rate Links. In *Proc. of IFIP Networking Conference*. IEEE, New York, NY, USA, 9.

[9] Mike Kosek, Hendrik Cech, Vaibhav Bajpai, and J"org Ott. 2022. Exploring Proxying QUIC and HTTP/3 for Satellite Communication. In *Proc. of IFIP Networking Conference*. IEEE, New York, NY, USA, 1–9. https://doi.org/10.23919/IFIPNetworking55013.2022.9829773

[10] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. 2019. Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. In *Proc. of the 26th Annual Network and Distributed System Security Symposium (NDSS 2019)*. https://doi.org/10.14722/ndss.2019.23386

[11] Gonzalo Martinez, Jose Alberto Hernandez, Pedro Reviriego, and Paul Reinheimer. 2023. Round Trip Time (RTT) Delay in the Internet: Analysis and Trends. *IEEE Network* (2023), 1–6. https://doi.org/10.1109/MNET004.2300008

[12] Robin Marx, Joris Herbots, Wim Lamotte, and Peter Quax. 2020. Same Standards, Different Decisions: A Study of QUIC and HTTP/3 Implementation Diversity. In *Proc. of EPIQ*. ACM, New York, NY, USA, 14–20. https://doi.org/10.1145/3405796.3405828

[13] Robin Marx, Wim Lamotte, Jonas Reynders, Kevin Pittevils, and Peter Quax. 2018. Towards QUIC debuggability. In *Proc. of the Workshop on the Evolution, Performance, and Interoperability of QUIC, EPIQ@CoNEXT 2018, Heraklion, Greece, December 4, 2018*. ACM, 1–7. https://doi.org/10.1145/3284850.3284851

[14] Robin Marx, Luca Niccolini, Marten Seemann, and Lucas Pardue. 2023. *Main logging schema for qlog*. Internet-Draft – work in progress 06. IETF. https://datatracker.ietf.org/doc/html/draft-ietf-quic-qlog-main-schema-06

[15] Ayush Mishra, Sherman Lim, and Ben Leong. 2022. Understanding Speciation in QUIC Congestion Control. In *Proc. of ACM IMC* (Nice, France) *(IMC '22)*. ACM, New York, NY, USA, 560—-566. https://doi.org/10.1145/3517745.3561459

[16] Mohamed Moulay, Fernando Díez Muñoz, and Vincenzo Mancuso. 2021. On the Experimental Assessment of QUIC and Congestion Control Schemes in Cellular Networks. In *Proc. of IEEE MedComNet*. IEEE, New York, NY, USA, 1–8. https://doi.org/10.1109/MedComNet52149.2021.9501271

[17] Marcin Nawrocki, Raphael Hiesgen, Thomas C. Schmidt, and Matthias W ählisch. 2021. QUICsand: Quantifying QUIC Reconnaissance Scans and DoS Flooding Events. In *Proc. of ACM IMC*. ACM, New York, NY, USA, 283–291. https://doi.org/10.1145/3487552.3487840

[18] Marcin Nawrocki, Pouyan Fotouhi Tehrani, Raphael Hiesgen, Jonas Mücke, Thomas C. Schmidt, and Matthias Wählisch. 2022. On the interplay between TLS certificates and QUIC performance. In *Proc. of ACM CoNEXT*. ACM, New York, NY, USA, 204–213. https://doi.org/10.1145/3555050.3569123

[19] Jan Rüth, Ingmar Poese, Christoph Dietzel, and Oliver Hohlfeld. 2018. A First Look at QUIC in the Wild. In *Passive and Active Measurement (LNCS, Vol. 10771)*. Springer Nature, Switzerland, 255–268.

[20] Darius Saif, Chung-Horng Lung, and Ashraf Matrawy. 2021. An Early Benchmark of Quality of Experience Between HTTP/2 and HTTP/3 using Lighthouse. In *Proc. of IEEE ICC*. IEEE, New York, NY, USA, 1–6. https://doi.org/10.1109/ICC42927.2021.9500258

[21] Constantin Sander, Ike Kunze, and Klaus Wehrle. 2022. Analyzing the Influence of Resource Prioritization on HTTP/3 HOL Blocking and Performance. In *6th Network Traffic Measurement and Analysis Conference, TMA 2022, Enschede, The Netherlands, June 27-30, 2022*, Roya Ensafi, Andra Lutu, Anna Sperotto, and Roland van Rijswijk -Deij (Eds.). IFIP. https://dl.ifip.org/db/conf/tma/tma2022/tma2022-paper28.pdf

[22] Marten Seemann and Jana Iyengar. 2020. Automating QUIC Interoperability Testing. In *Proc. of EPIQ* (Virtual Event, USA) *(EPIQ '20)*. ACM, New York, NY, USA, 8–13. https://doi.org/10.1145/3405796.3405826

[23] Tanya Shreedhar, Rohit Panda, Sergey Podanev, and Vaibhav Bajpai. 2021. Evaluating QUIC Performance over Web, Cloud Storage and Video Workloads. *IEEE Transactions on Network and Service Management* (2021), 16 pages. https://doi.org/10.1109/TNSM.2021.3134562

[24] Tanya Shreedhar, Rohit Panda, Sergey Podanev, and Vaibhav Bajpai. 2022. Evaluating QUIC Performance Over Web, Cloud Storage, and Video Workloads. *IEEE Transactions on Network and Service Management* 19, 2 (jun 2022), 1366–1381. https://doi.org/10.1109/TNSM.2021.3134562

[25] Douglas Stebila and Nick Sullivan. 2015. An Analysis of TLS Handshake Proxying. In *Proc. of IEEE Trustcom*. IEEE, New York, NY, USA, 279–286. https://doi.org/10.1109/Trustcom.2015.385

[26] Martino Trevisan, Danilo Giordano, Idilio Drago, Marco Mellia, and Maurizio M. Munafò. 2018. Five years at the edge: watching internet from the ISP network. In *Proc. of the 14th International Conference on emerging Networking EXperiments and Technologies, CoNEXT 2018, Heraklion, Greece, December 04-07, 2018*, Xenofontas A. Dimitropoulos, Alberto Dainotti, Laurent Vanbever, and Theophilus Benson (Eds.). ACM, 1–12. https://doi.org/10.1145/3281411.3281433

[27] Konrad Wolsing, Jan Rüth, Klaus Wehrle, and Oliver Hohlfeld. 2019. A Performance Perspective on Web Optimized Protocol Stacks: TCP+TLS+HTTP/2 vs. QUIC. In *Proc. of the Applied Networking Research Workshop (ANRW '19)*. ACM, New York, NY, USA, 1–7. https://doi.org/10.1145/3340301.3341123

[28] Xiangrui Yang, Lars Eggert, Jörg Ott, Steve Uhlig, Zhigang Sun, and Gianni Antichi. 2020. Making QUIC Quicker With NIC Offload. In *Proc. of the 2020 Workshop on the Evolution, Performance, and Interoperability of QUIC, EPIQ@SIGCOMM 2020, Virtual Event, USA, August 10-14, 2020*, Jörg Ott and Lars Eggert (Eds.). ACM, 21–27. https://doi.org/10.1145/3405796.3405827

[29] Alexander Yu and Theophilus A. Benson. 2021. Dissecting Performance of Production QUIC. In *Proc. of the Web Conference* (Ljubljana, Slovenia) *(WWW '21)*. ACM, New York, NY, USA, 1157—-1168. https://doi.org/10.1145/3442381.3450103

[30] Johannes Zirngibl, Philippe Buschmann, Patrick Sattler, Benedikt Jaeger, Juliane Aulbach, and Georg Carle. 2021. It's over 9000: Analyzing Early QUIC Deployments with the Standardization on the Horizon. In *Proc. of ACM IMC*. ACM, New York, NY, USA, 261–275. https://doi.org/10.1145/3487552.3487826

[31] Johannes Zirngibl, Florian Gebauer, Patrick Sattler, Markus Sosnowski, and Georg Carle. 2024. QUIC Hunter: Finding QUIC Deployments and Identifying Server Libraries Across the Internet. In *Passive and Active Measurement - 25th International Conference, PAM 2024, Virtual Event, March 11-13, 2024, Proceedings, Part II* , Philipp Richter, Vaibhav Bajpai, and Esteban Carisimo (Eds.). Springer, 273–290. https://doi.org/10.1007/978-3-031-56252-5_13

A Ethics

This work does not raise any ethical issues.

B Artifacts

We publish all artifacts of this paper, raw data, processing, and setup scripts.

Data set: Qlog logs, packet captures, QScanner [30] results

Run-time environment: Python, Jupyter Notebooks

How much disk space is required? ~600 GB

How much memory is required? 256 GB

Publicly available? Yes

Archived? Yes: https://doi.org/10.5281/zenodo.13743301

Our emulation uses Docker images pulled on 06.07.2024, except for *mvfst*, which was pulled on 19.08.2024 to include a more recent Docker image compared to the start of our measurement.

C Guidelines for Instant ACK Deployment

Certificate size and the delay between frontend and certificate store vary between different deployments. Table 2 summarizes when WFC and IACK should be preferred. If the certificate size causes the handshake to exceed the anti-amplification limit, enabling instant ACK allows the server a higher sending budget and will improve performance. Further, if the delay between frontend and certificate

Table 2: Deployment suggestions with and without packet loss. In the majority of scenarios, using instant ACK is advised.

Certificate size vs. amplification limit	Loss		No Loss	
	First server flight except first datagram	Second client flight	Frontend-certificate store delay Δt	
			Δt < 3 RTT (PTO)	$\Delta t \geq$ 3 RTT (PTO)
(1) certificate size ≤ anti-amplification limit	WFC	IACK	IACK	WFC
(2) certificate size > anti-amplification limit	IACK	IACK	IACK	IACK

store is smaller than the client PTO (3× RTT), the client will react earlier to packet loss events. If instead, the server default PTO is important, *e.g.,* the first server flight, except the first datagram, is lost, an instant ACK does not provide the server with an RTT sample, which will slow server resends. Last, if the frontend to certificate store delay is larger than the client PTO an instant ACK causes futile load on the server, which can decrease performance but is often tolerable by CDN deployments. Optimally, servers should adjust the utilization of instant ACK depending on the expected certificate size and current frontend to certificate store delay.

D ACK Delay: An Alternative to Instant ACK?

In this section, we explain in detail why instant ACK cannot be completely replaced by using `ACK Delay`, a dedicated field in ACK frames to encode the acknowledgment delay [7]. ACK Delay is not sufficient for three reasons. First, the PTO initialization ignores the acknowledgment delay of the first RTT sample. As such it would only help to reinitialize the PTO with the second sample. Second, current server implementations often communicate an ACK Delay of 0. Third, current CDN deployment challenge the use.

Limited support in server implementations. We verify the support of ACK Delay by collecting the packet captures of successful handshake tests provided by the public QIR covering the 14 supported server implementations (*i.e., aioquic, go-x-net, haproxy, kwik, lsquic, msquic, mvfst, neqo, nginx, ngtcp2, picoquic, quic-go, quiche, quinn, s2n, xquic*), using a quic-go client from three subsequent days in August 2024.

Table 3 indicates that 6 implementations report an ACK Delay of 0 ms in the first packet. *msquic* does not send Initial and Handshake ACKs. The remaining implementations report delays between 0.4 ms (*quinn*) and 15.2 ms (*s2n-quic*). The reported delay of *s2n-quic* exceeds the RTT of the connection. 11 implementations do not send an acknowledgment in the Handshake name space. 3 implementations report an acknowledgment delay larger than 0 ms.

Challenges in the wild. Figure 10 shows that the acknowledgment delay exceeds the RTT in most responses with coalesced ACK–SH from Akamai (99.8%), Amazon (87.3%), Cloudflare (99.9%), Fastly (60.5%) and Meta (100%) and Others (77.9%). For Google, we observe that this is the case in 34.8%. However, the difference between RTT and acknowledgment delay is less than 1ms for 99.8% of the domains across all CDNs. On the contrary, the majority of IACKs carry an acknowledgment delay larger than the RTT, except for Akamai and Others, where 61% and 79.1% are below that threshold.

According to the standard, in Initial packets clients may ignore the acknowledgment delay and in any packet number space, the acknowledgment delay should not be subtracted from the RTT sample, if the resulting value is lower than *min_rtt* [6]. The consequence of sending such high acknowledgment delays in coalesced ACK–SH can lead to clients ignoring them or significantly underestimating the actual path RTT. Current implementations challenge the use of this alternative.

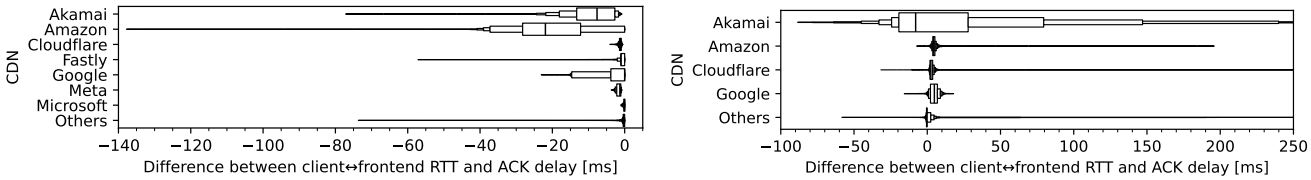
Table 3: Delay of the first acknowledgment received from server in the Initial and Handshake packet number space.

	First ACK Delay [ms]					
	Repetition [index]					
	1	2	3	1	2	3
Server	Initial Packet			Handshake Packet		
aioquic	3.3	3.4	3.3	-	-	-
go-x-net	0.0	0.0	-	-	-	-
haproxy	1.0	1.0	-	0.0	0.0	-
kwik	0.0	0.0	0.0	-	-	-
lsquic	1.2	1.1	1.2	0.2	0.2	0.2
msquic	-	-	-	-	-	-
mvfst	0.8	-	0.7	0.2	-	0.1
neqo	0.0	0.0	-	0.0	0.0	-
nginx	0.0	0.0	0.0	-	-	-
ngtcp2	0.0	0.0	0.0	-	-	-
picoquic	0.8	0.7	0.8	-	-	-
quic-go	0.0	0.0	0.0	-	-	-
quiche	1.4	1.4	1.5	-	-	-
quinn	0.4	-	0.4	-	-	-
s2n-quic	14.0	15.2	14.1	-	-	-
xquic	1.3	1.1	1.2	-	0.5	0.5

E Client Implementation Details

Default PTO. RFC 9000 recommends a default PTO of 1 s. The majority of the analyzed implementations deviate significantly from the default to optimize recovery in case of packet loss. Table 4 shows the designated default PTO by implementation.

Packet coalescence. To analyze the performance impact of the instant ACK, we analyze QUIC connections with and without packet loss. QUIC allows coalescence of multiple QUIC packets in one datagram, *i.e.,* the information exchanged in one datagram varies. An unbiased experiment should compare equal information loss. We analyze the content of each packet exchanged during the handshake and find that implementations coalesce QUIC packets differently but generally consistent, *e.g.,* the second client flight (see Figure 3) is transmitted in 1 to 4 datagrams. We account for these differences by comparing loss of equal information. Hence, when analyzing loss of



(a) Coelesced ACK–SH **(b) IACK**

Figure 10: Difference between RTT and acknowledgment delay. Coalesced ACK–SHs tend to carry an acknowledgment close to or exceeding the RTT. IACKs more frequently contain values lower than the RTT, allowing the client to correctly adjust the RTT sample.

Table 4: Initial PTO and UDP datagrams comprising the second client flight. Implementations chose lower initial PTOs than the recommended value of 1 s [6] to improve recovery from packet loss. Due to packet coalescence the second client flight is sent in different UDP datagrams.

Implementation	Default PTO [ms]	Second flight datagram(s) [index]
aioquic	200	2,3,4
go-x-net	999	2,3,4
mvfst	100	2,3,4
neqo	300	2,3
ngtcp2	300	2,3,4
picoquic	250	2,3,4,5
quic-go	200	2,3,4
quiche	999	2

the second client flight, we respect the individual packet coalescence and compare loss of different datagrams. Table 4 specifies which datagrams this affects. In case clients send additional packets (*e.g.,* when the default PTO expires), this static mapping no longer drops the entire second client flight (*cf.,* Figure 13, 300 ms RTT).

Standard conformance and limitations of Qlog. The transferability of our results to actual deployments, depends on the conformance of implementations to the standard and precision of information exposed in Qlog. With respect to standard conformance, we find that *aioquic* uses a different formula to calculate RTT variance. Our calculation of PTOs is limited by the information exposed in Qlog. Within these logs, we find that timestamps are provided with different resolutions, *i.e.,* µs, ms, and s, and *neqo, mvfst* and *picoquic* do not log RTT variance. When RTT variance is not available, we calculate it from the sent and received packets instead.

Next, we compare the number of RTT samples considered by implementations with the number of theoretically possible RTT samples, *i.e.,* reception of acknowledgments for packets that are newly acknowledged. We approximate the number of RTT samples of implementations by the number of changes in smoothed RTT and RTT variation. When implementations log the same information multiple times, we remove consecutive duplicates. This post-processing step might remove distinct samples represented by the same numbers.

Figure 11 exhibits large differences in the number of RTT samples available to implementations. Implementations may send PING

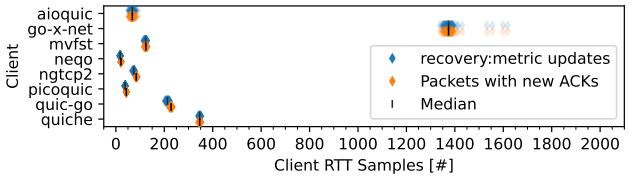


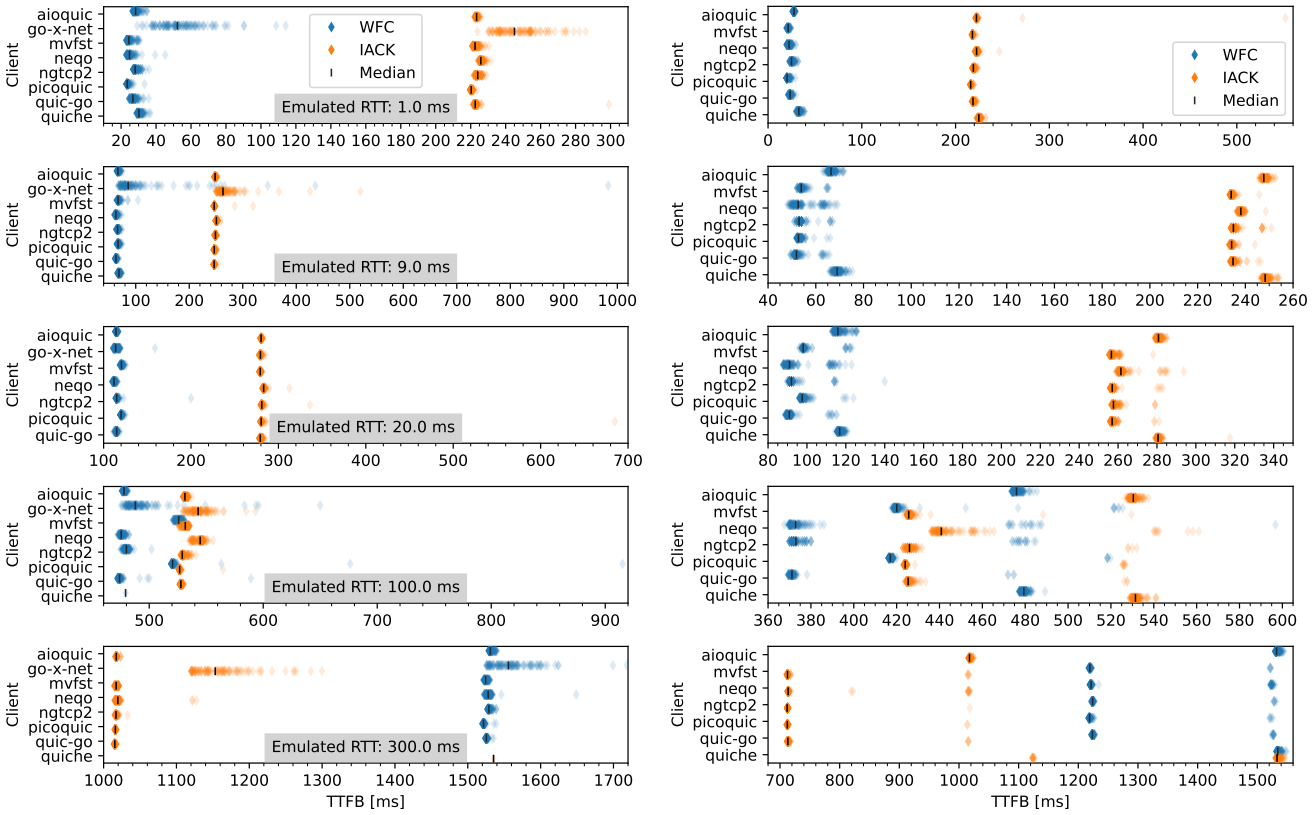
Figure 11: Number of exposed RTT samples and newly acknowledging ACKs for 10 MB file transfer at 100 ms RTT, WFC. Due to different use of ACK-eliciting packets, *e.g.,* PING frames, implementations vary in the amount of RTT samples they can obtain. They also expose different shares of the *recovery:metric* updates.

frames, which causes more RTT samples. Since the impact of the instant ACK on PTO declines and depends on the number of RTT samples, implementations with fewer RTT samples are influenced for a longer share of the connection. Further, we observe that *aioquic, go-x-net, mvfst*, and *quiche* expose the maximum of PTO updates available, while *neqo, ngtcp2, picoquic*, and *quic-go* rely on a smaller fraction of the samples. This indicates that an implementation (*i*) does not utilize all received ACKs or (*ii*) does not log all metric updates.

F Details on Packet Loss Scenarios

We repeat all packet loss emulations with 1, 9, 20, 100, and 300 ms RTT to ensure that our observations are valid across a wide range of network scenarios and compare the reception of the first payload byte (QUIC stream frame) after the loss event.

Time to First Byte is increased when the remaining first server flight is lost. Figure 12 shows that all QUIC connections until 100 ms RTT suffer from the server not holding an RTT estimate for the connection. At 100 ms RTT, the difference between IACK and WFC is reduced. In WFC, the TTFB is mainly influenced by the server receiving two RTT samples, (*i*) in the Initial and (*ii*) in the Handshake packet number space, which reduces the variance and leads to setting an earlier PTO. For IACK, the server default PTO expires before the client sends probe packets (exceptions are *mvfst, picoquic*). Only after an additional RTT, the client confirms the resent server Initials, which provides the server with a single RTT estimate that does not reduce the variance. A later Handshake PTO is set compared to WFC. For *mvfst* after 100 ms, in WFC the client default PTO expires and subsequently the server Handshake packet is acknowledged, which sets the Handshake PTO. In IACK, *mvfst*



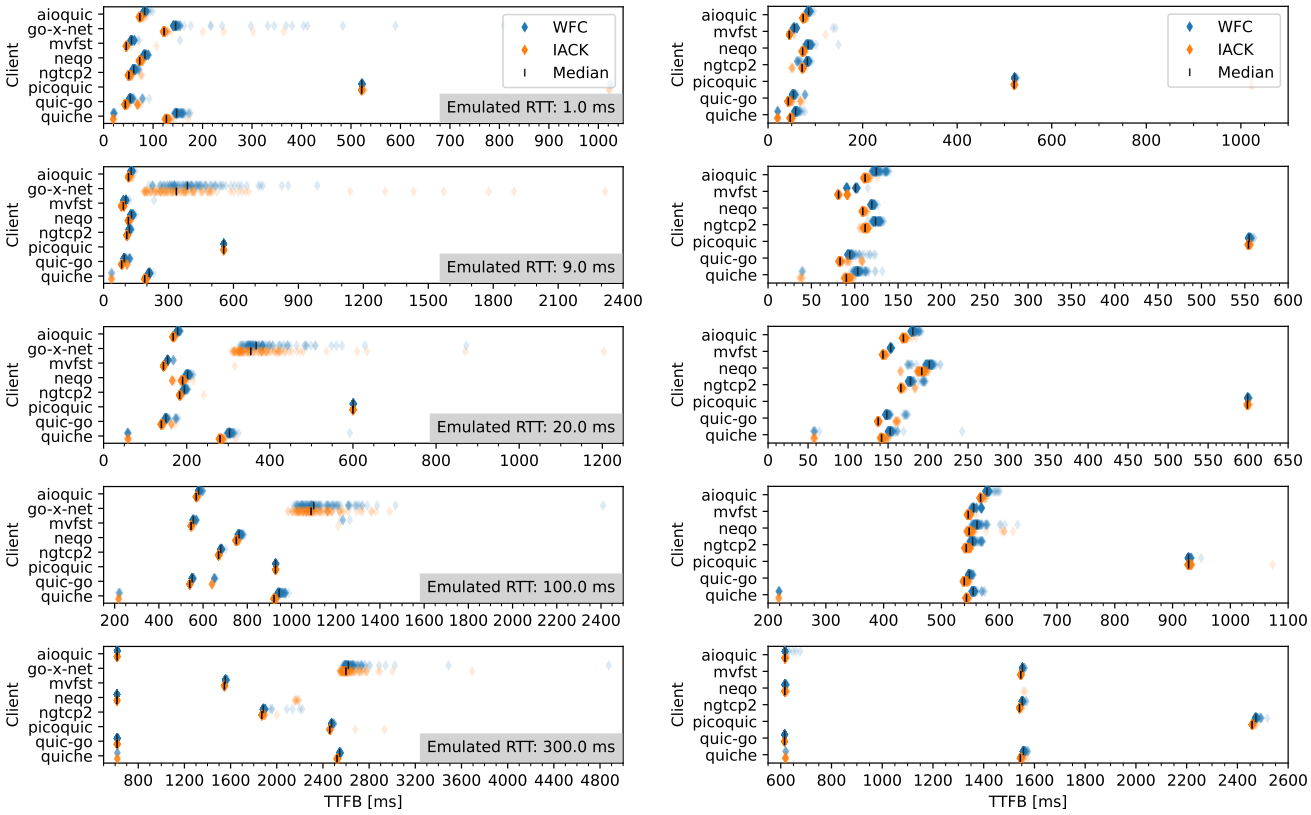
(a) HTTP/1 **(b) HTTP/3**

Figure 12: Time to First Byte of 10 KB file transfer at different RTTs under loss of packets 2 and 3 (IACK) and packet 2 (WFC) sent by the server. IACK prolongs the TTFB for all RTTs until the default PTO of the client is reached or until the PTO for the Handshake packet number space becomes relevant. Each row presents results from one emulated RTT.

does not expire the default PTO after reception of an instant ACK. Instead, the default PTO of the server expires and causes the client to acknowledge the retransmitted server packets, which provides the server with an RTT estimate. Both lead to similar timings for resending the server Handshake packets. For *picoquic*, in WFC the acknowledgment of the server Handshake packet provides the server with an RTT estimate. In IACK, *picoquic* does not send probe packets in response to the server instant ACK. Instead, the server PTO expires and picoquic computes the Handshake keys from the now received ServerHello and sends a Handshake PING packet, which causes the Handshake PTO to expire immediately. Both behaviors lead to similar TTFB.

At 300 ms RTT, IACK outperforms WFC due to restarting the PTO timer with a larger than default PTO after reception of an acknowledgment. In the case of IACK, the server only receives acknowledgments in the Initial packet number space, while in the case of WFC, the first packet contains a Handshake packet, which is acknowledged by the client. On reception of the Handshake ACK, the server sets a PTO based on the larger RTT for the outstanding Handshake packets, which in consequence delays the TTFB. The PTO is then larger than the default PTO of our server implementation, which leads to the differences. For *quiche*, at all RTTs the duplicate CID retirement issue caused abortion of the measurement immediately or after up to one successful exchange. Please note that the improvement of ≈200 ms originates from the default server PTO, a higher default server PTO will lead to a different advantage of WFC over IACK. Similar behavior is observed for HTTP/3.

Time to First Byte is reduced when the second client flight is lost. Figure 13 shows a general improvement of IACK over WFC for all RTTs. *picoquic* does not account for the shorter RTT estimate with IACK and relies solely on its default PTO. The PTO timer is reset after a retransmission from the server is received, due to the server default PTO expiring in IACK and WFC. For *quiche*, two kinds of behaviors are observable, the lighter diamonds at an earlier time originate from sending two packets instead of one packet for the client second flight. Only one packet is dropped by our emulation (see Table 4). The server then already holds the client request and is able to serve it sooner. At 300 ms RTT for *aioquic, mvfst, neqo, ngtcp2* and *quic-go* the default PTO expires before any further packet from the server is processed and the second client flight can be sent. Instead, the ClientHello is deemed lost and resent. Our static mapping of the second client flight then drops the probe packets, which cause the HTTP request to arrive earlier (see Table 4). *quiche* and *go-x-net* use a large default PTO



(a) HTTP/1 (b) HTTP/3

Figure 13: Time to First Byte of 10 KB file transfer at different RTTs under loss of the entire client second flight. IACK improves the TTFB. Each row presents results from one emulated RTT.

Table 5: AS numbers used for CDN inferences.

CDN	AS numbers
Akamai	16625, 20940
Amazon	14618, 16509
Cloudflare	13335, 209242
Fastly	54113
Google	15169, 396982
Meta	32934
Microsoft	8075

and are thus not impacted by probe packet drop instead of second client flight drop. Similar behavior can be observed for our HTTP/3 measurement.

G Macroscopic View

Which QUIC implementation is deployed by a CDN? Different behavior of CDNs may originate from different implementations (*e.g.,* Cloudflare deploys *quiche* with IACK, while Meta deploys *mvfst* without IACK). Zirngibl *et al.* [31] show that the order of QUIC transport parameters and the content of `CONNECTION_CLOSE` frames can be used to identify server implementations. Identifying different QUIC stacks is beyond the scope of this work, but applying such a method could be part of future work.

Assignment of IP addresses to on-net CDN deployments. CDN hosted domains are inferred from their IP addresses mapped to origin ASes gained from route announcements on August 7, 2024. To account for CDNs operating multiple ASes, we assign multiple AS numbers to one CDN according to Table 5.

CDN deployment variations. Figure 14 exhibits homogeneous delays between instant ACK and later ServerHello from Akamai, Amazon, Cloudflare, and Others from all four measurement locations. Google IACK-enabled servers are only significantly reachable from Sao Paulo. Otherwise, implementations vary in the share of IACK deployment by up to 8.3% between different days from the same location.

Cross-verification of Cloudflare IACK deployment. For cross-verification, we performed the measurement from a European university network (Hamburg, DE) and Google Cloud in North America (Los Angeles, US), South America (Sao Paulo, BR) and Asia (Hong Kong, HK) in August 2024. Figure 15 shows the median time between the request and response packet from Cloudflare servers at the same location as our probe. At all locations the coalesced ACK–SH is faster than the separated ServerHello. Gaps

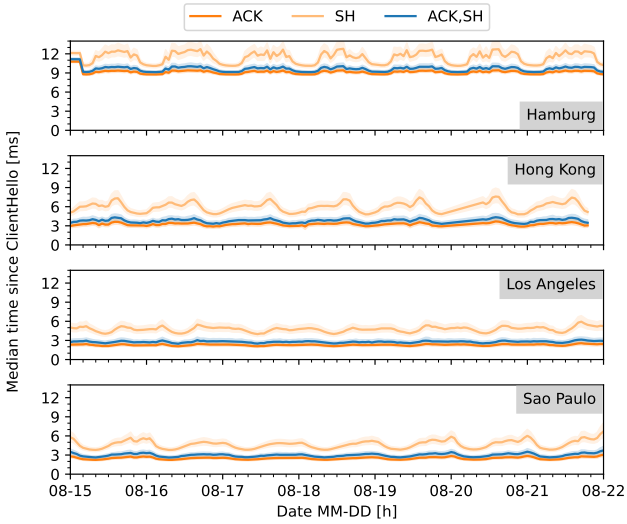


Figure 15: Time between request and response from Cloudflare servers from the measurement locations with 50 % percentile interval. The gaps in the measurements from Hong Kong are caused by a misconfiguration of our nodes.

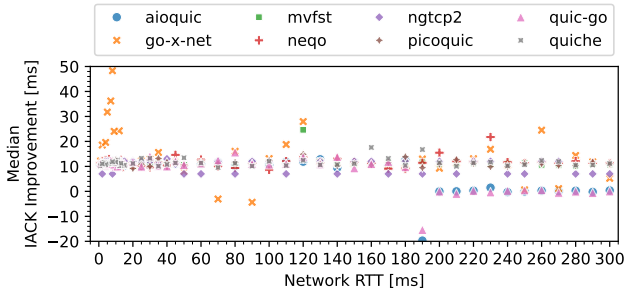


Figure 16: Improvement of the first PTO, based on recovery metric updates in Qlog. The variance is calculated from the logged packet receptions, if it is not provided by the implementation.

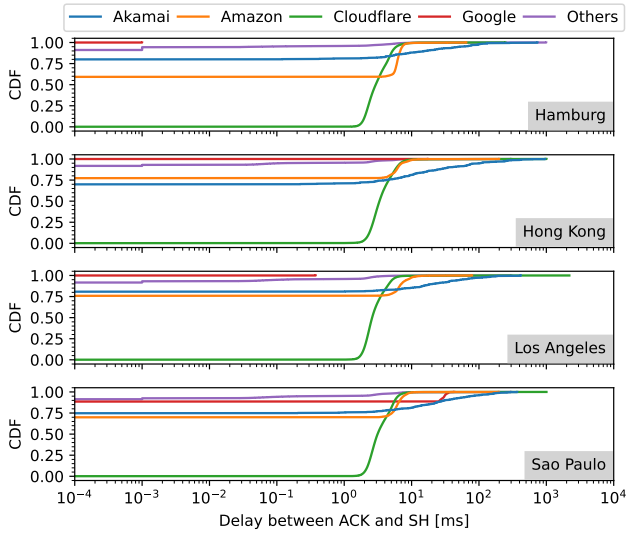


Figure 14: Delay between reception of the first ACK and subsequent ServerHello (SH) from our four vantage points for domains on the Tranco Top 1M. IACK performance is similar across locations.

in the measurement from our vantage point are caused by host maintenance temporarily deactivating our virtual machine.

IACK provides a consistent PTO improvement across RTTs. Figure 16 shows the median PTO improvement of IACK over WFC derived from the first smoothed RTT and RTT variance information provided by the implementations. If the variance is not provided, we calculate it based on packet reception times. Implementations exhibit similar PTO improvements across all RTTs. *go-x-net* partially initializes the PTO incorrectly, which also influence the median difference. Beginning at 190 ms RTT, *quic-go* and *aioquic* only log RTT estimates after multiple RTTs. This correlates with their default PTO expiring at that same time.